

项目拆解-核心代码

项目完全拆解手册

简历写法（参考-可自己选择职责和成果添加）

企业级多Agent深度研究智能助手 DeepResearch

项目介绍：在企业级深度研究场景中，分析师面临信息分散在多个平台、需要交叉验证、人工检索耗时且难以保证全面性等痛点；同时生成式 AI 直接回答存在"幻觉"风险，无法提供可追溯的引用来源。为此搭建基于多 Agent 协作的 AI 深度研究平台，通过意图路由自动识别研究型查询，调度多专家 Agent 并行检索网络与本地知识库，经证据审计与迭代补搜后生成带精确引用的深度研报，实现从"人找信息"到"AI 自动研究"的范式转变。

技术栈：Python · FastAPI · LangGraph · LangChain · Milvus · PostgreSQL · Qwen

职责描述：

- 多 Agent 协作编排：**基于 LangGraph 状态机构建 8 大专家 Agent（意图路由/规划/网络侦察/本地侦察/证据裁判/分析/反思/撰稿），通过共享状态ResearchState实现上下文流转；意图路由采用规则引擎+大模型双模态判断，区分"闲聊/简单问答→直接回答"和"调研/分析→深度研究"两种链路。
- 双源检索与证据审计：**构建"网络搜索 + 本地知识库"双检索链路，并行执行网络侦察与本地侦察；证据裁判基于信源类型评分，实现去重、冲突检测与审计标记，从海量原始结果中筛选出高质量证据。
- 迭代式研究闭环：**分析 Agent 基于证据池生成研究结论并评估证据完备性，反思 Agent 针对信息缺口生成补充查询触发补搜迭代；通过最大迭代次数与预算控制研究深度，实现"证据不足→补搜→再分析"的递归优化闭环。
- 引用溯源与幻觉防控：**撰稿 Agent 生成研报时强制使用合法来源编号，通过正则校验引用合法性，自动移除幻觉引用；引用列表自动渲染，支持网络/本地双类型溯源。
- 三层记忆系统：**设计短期记忆（PostgreSQL，会话级）、长期记忆（PostgreSQL，用户级）、语义记忆（Milvus，向量级）三层记忆架构；记忆管理器统一封装保存上下文/检索上下文/语义搜索接口，支持跨会话上下文继承与个性化应答。

项目成果（供选择，可自己调整）：

【质量指标】

- ▶ 基于 8-Agent 协作与迭代补搜机制，在内部测评集（约 200 条复杂查询）上，回答幻觉率由约 25% 降至 6%，引用准确率达到 94%。
- ▶ 系统支持多轮迭代补搜，在证据不足场景下通过反思 Agent 自动生成补充查询，研究完备性满意度由 62% 提升至 89%。
- ▶ 引入证据评分与冲突检测机制后，低质量信源（评分<0.6）占比从 45% 降至 12%，有效过滤了自媒体与论坛中的不可靠信息。

【性能指标】

- ▶ 双源检索链路平均响应时间 < 8s，相比单源检索证据覆盖率提升 40%；证据评分机制使高质量信源占比从 35% 提升至 78%。
- ▶ 通过并行执行网络侦察与本地侦察，检索阶段耗时降低 35%，端到端研报生成时间控制在半小时以内（由原先的小时级别缩短）。
- ▶ 引入规则引擎预筛选后，意图路由准确率达到 96%，相比纯大模型判断减少约 60% 的无效深度研究调用。

一、项目背景与定位

1.1 行业背景

在 AI 大模型时代，用户对于信息获取的需求已经从简单的问答升级为**深度研究**和**知识整合**。传统的单轮对话模式无法满足以下场景：

- **投资研究**：需要分析某个行业趋势，需要多源数据交叉验证
- **技术调研**：需要对比多个技术方案，查看官方文档和社区讨论
- **竞品分析**：需要收集竞争对手的产品信息、用户评价、市场份额
- **政策解读**：需要追踪政策来源、官方解读、专家分析

这些场景的共同特点是：

1. **信息分散**：答案不在单一来源，需要多处检索

- 2. 需要验证：信息需要交叉验证，识别冲突和谣言
- 3. 逻辑复杂：需要多步推理，形成完整结论
- 4. 可追溯性：需要明确的引用来源，支持事实核查

1.2 产品定位

是一款面向企业级深度研究场景**的 AI 多智能体系统，定位为 "AI 研究员助手"。

核心能力矩阵：

能力维度	传统 Chatbot	Multi-Agents Memory
信息来源	训练数据（静态）	实时检索 + 本地知识库
回答长度	简短回复	3000+ 字深度报告
引用溯源	无	精确到来源 ID
冲突检测	无	自动识别并标记
迭代优化	单轮	多轮补搜直到完备
记忆继承	单会话	跨会话长期记忆

目标用户：

- 投资分析师、行业研究员
- 技术架构师、产品经理
- 咨询顾问、政策分析师
- 知识管理专员

1.3 技术演进路线

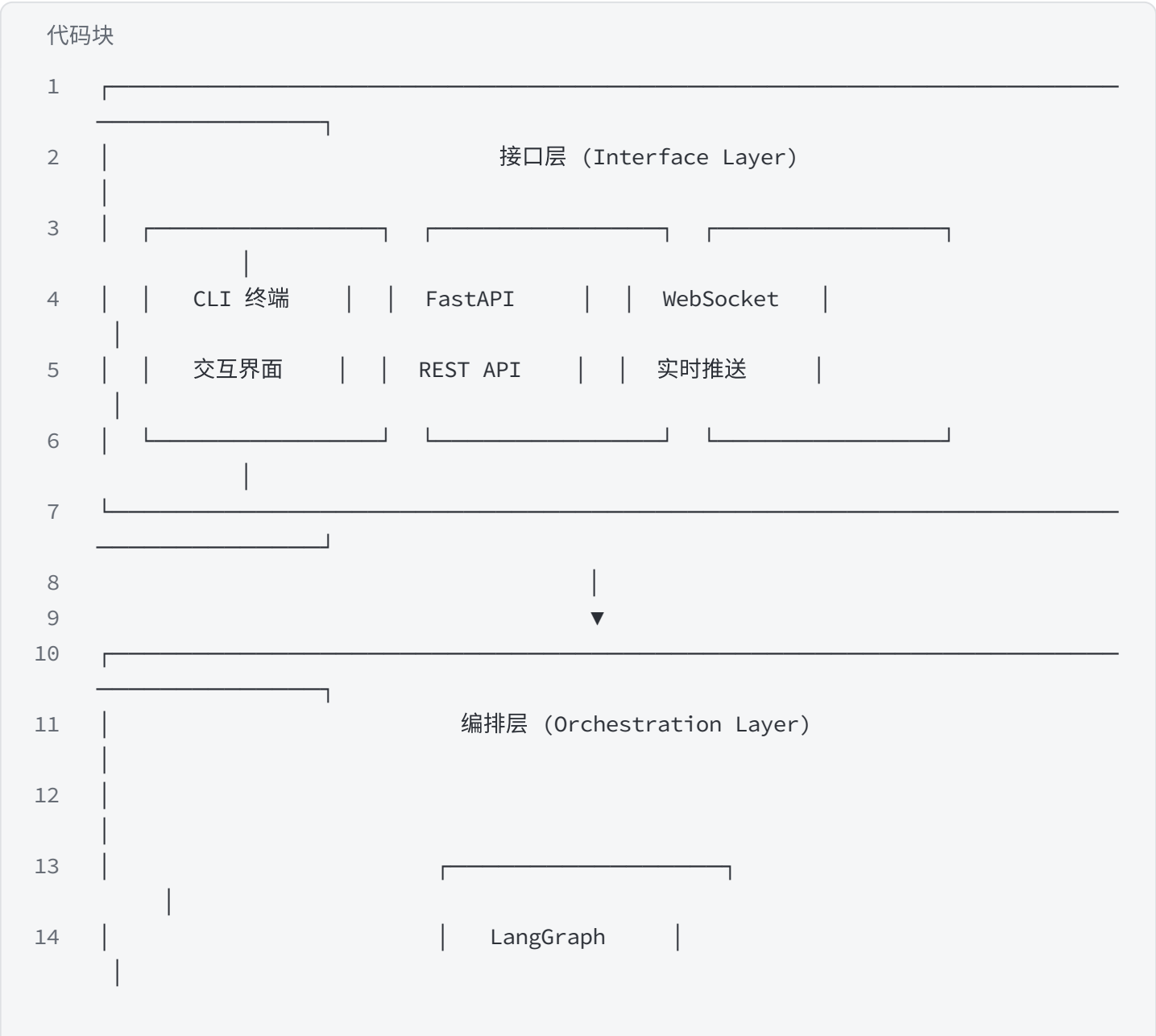
代码块

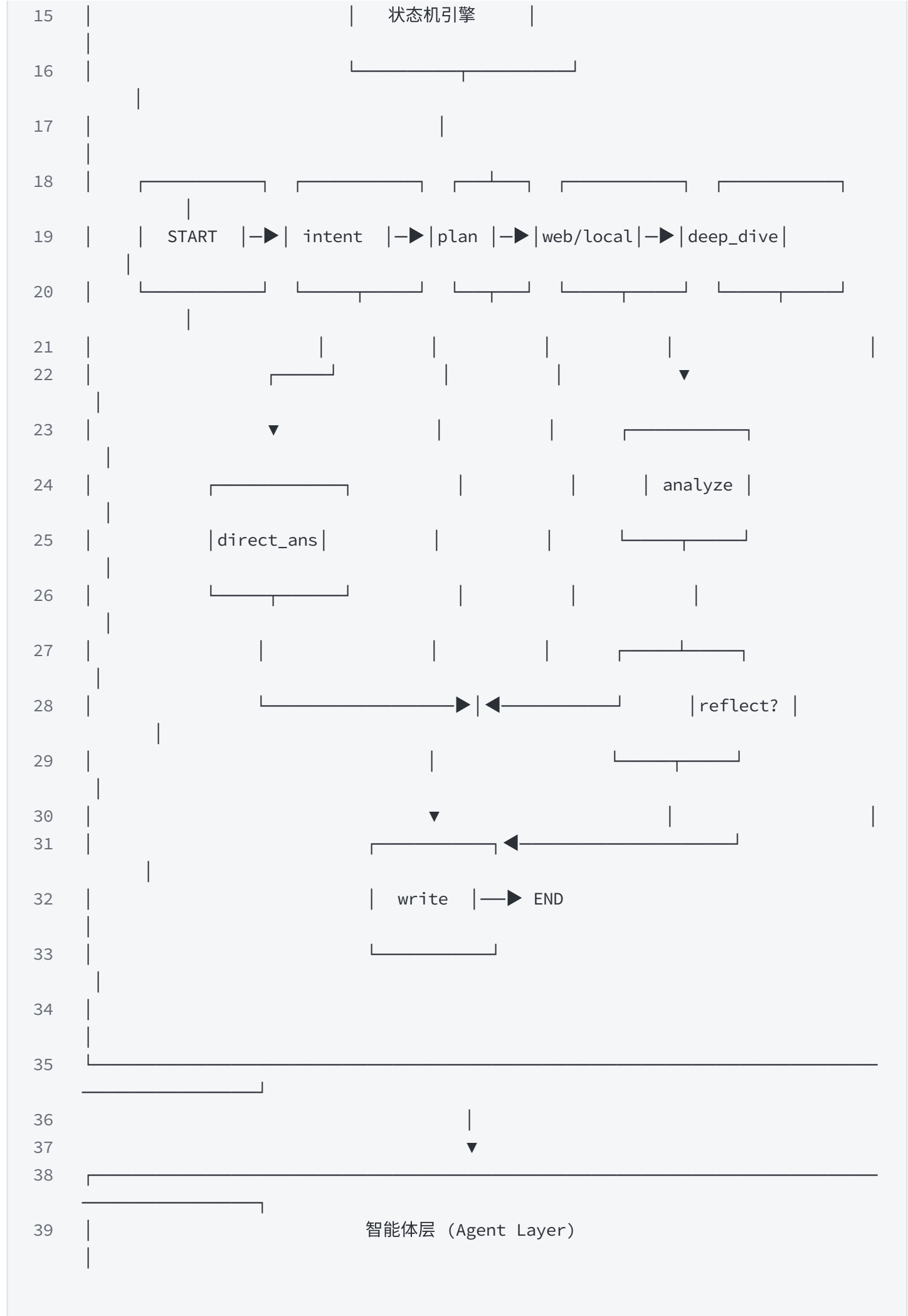
```
1  V1.0 单 Agent RAG
2      |
3      ▼ 问题：无法处理复杂多跳查询
4  V2.0 多 Agent 协作（当前版本）
5      |
6      └─ Intent Router：智能路由
```

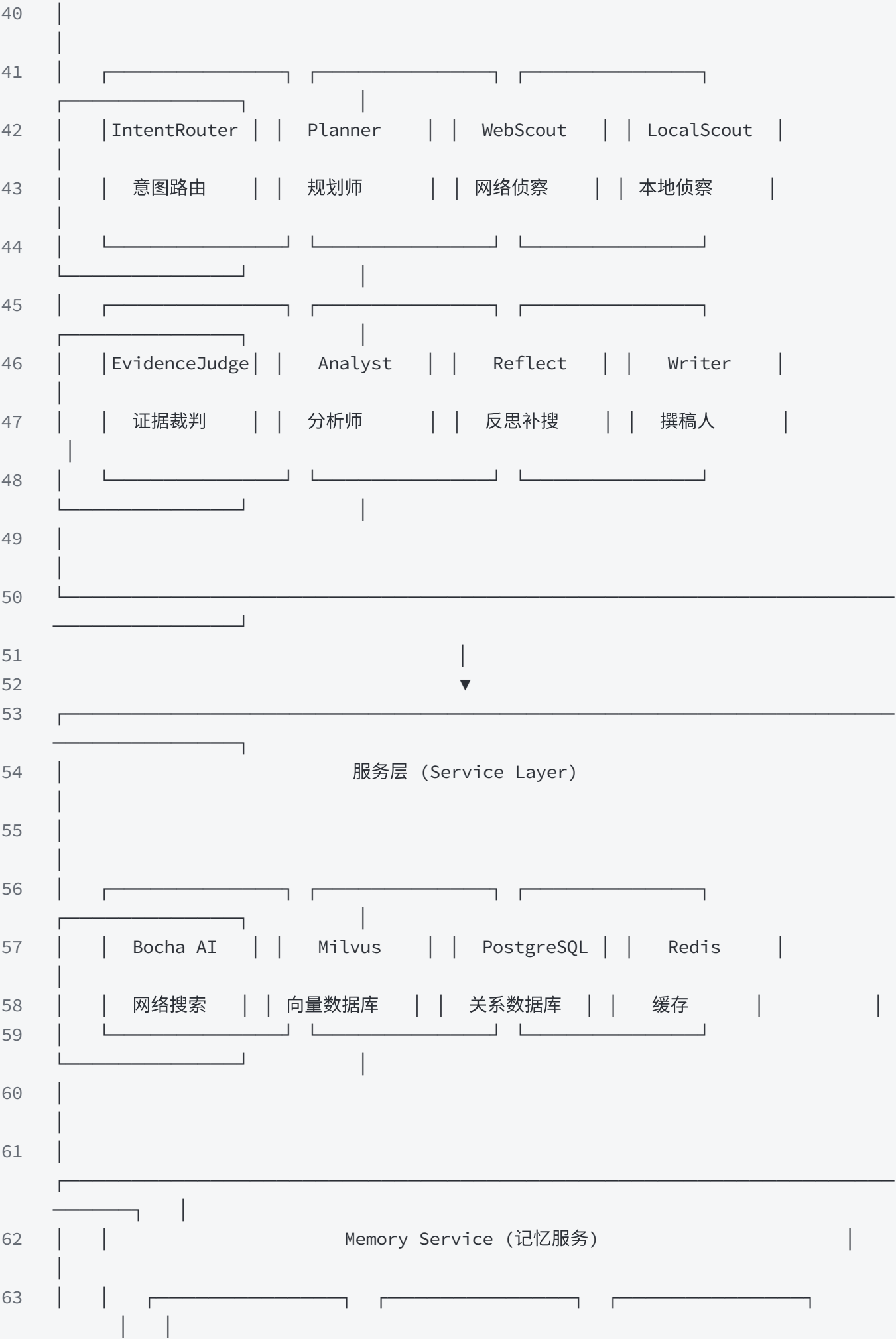
- 7 |— Planner: 任务拆解
- 8 |— Scout (Web/Local): 双源检索
- 9 |— Evidence Judge: 证据裁判
- 10 |— Analyst: 分析归纳
- 11 |— Reflect: 迭代补搜
- 12 |— Writer: 报告撰写
- 13

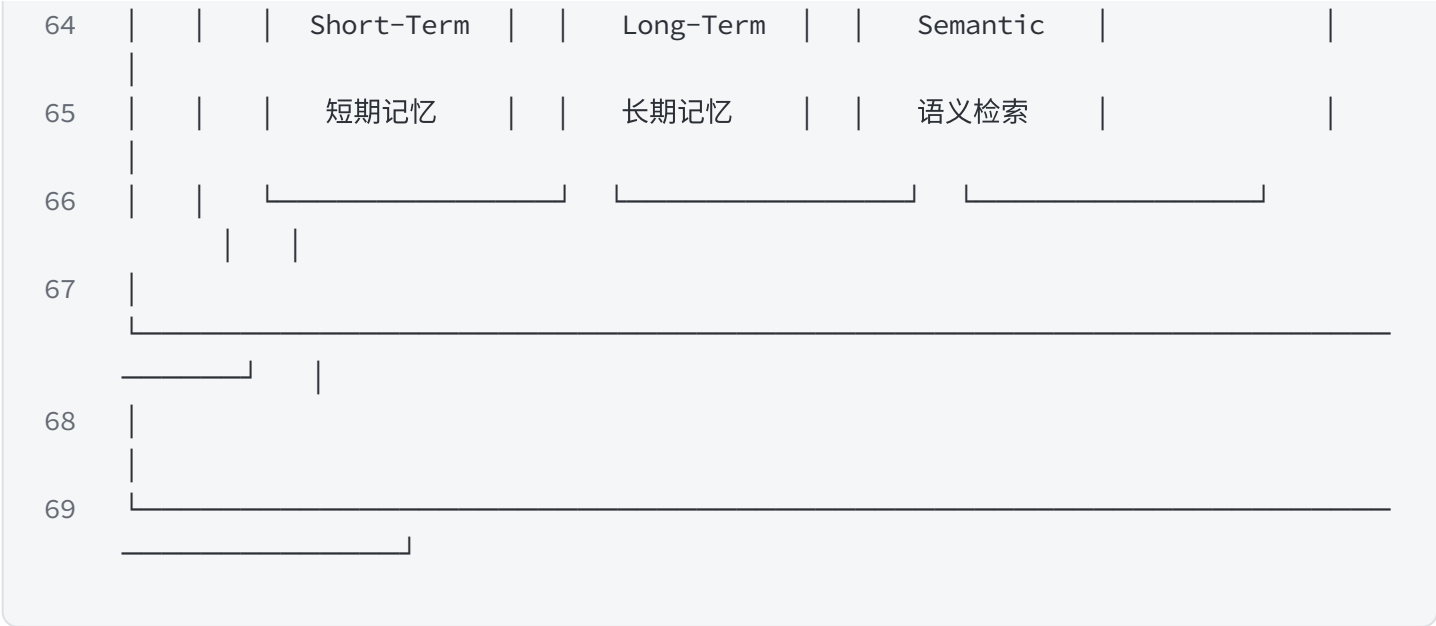
二、系统架构详解

2.1 分层架构图





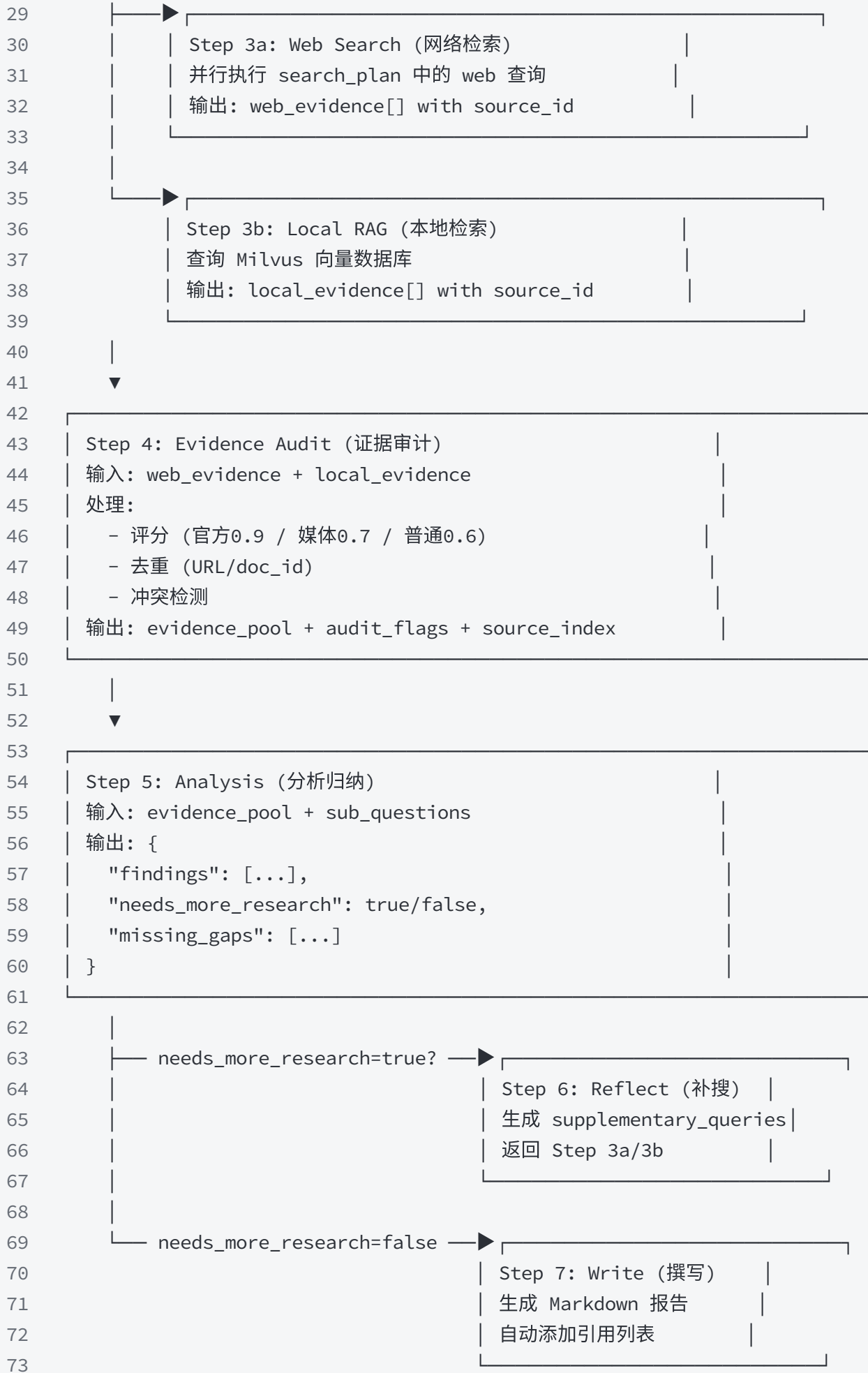




2.2 核心数据流

代码块

```
1  用户输入 Query
2    |
3    ▼
4  [ Step 1: Intent Recognition (意图识别)
5    | 输入: "帮我调查2026年最好的AI产品"
6    | 输出: {"route": "multiagent", "reason": "需要调研和盘点"}
7    |
8  ]
9    |
10   ▼
11 [ Step 2: Task Planning (任务规划)
12 | 输入: 原始问题
13 | 输出: {
14 |   "objective": "调查2026年最佳AI产品",
15 |   "sub_questions": [
16 |     "2026年AI产品市场概况",
17 |     "各细分领域最佳产品",
18 |     "用户评价和专业评测"
19 |   ],
20 |   "outline": [...],
21 |   "search_plan": [
22 |     {"query": "2026 best AI products", ...},
23 |     {"query": "AI product reviews 2026", ...}
24 |   ]
25 | }
26 |
27 ]
28 |
```



三、Agent 完全拆解

3.1 Agent 构建机制

核心代码（main.py）：

代码块

```
1  @dataclass(frozen=True)
2  class AgentBundle:
3      """Agent 集合，每个 Agent 是一个独立的 LLM 实例"""
4      intent_router: any          # 意图路由器
5      planner: any                # 规划师
6      scout_web: any              # 网络侦察员
7      scout_local: any           # 本地侦察员
8      evidence_judge: any         # 证据裁判
9      analyst: any                # 分析师
10     direct_responder: any       # 直接回答器
11     writer: any                  # 撰稿人
12
13     def build_agent(model: str, api_key: str, prompt_key: str, temperature: float,
14                     tools: list):
15         """
16         构建单个 Agent
17
18         Args:
19             model: 模型名称，如 "qwen-turbo"
20             api_key: DashScope API Key
21             prompt_key: PROMPTS 字典的 key
22             temperature: 温度参数（创意性 vs 确定性）
23             tools: 绑定的工具列表
24         """
25         if api_key:
26             os.environ["DASHSCOPE_API_KEY"] = api_key
27             llm = ChatTongyi(model=model, temperature=temperature)
28             prompt = PROMPTS[prompt_key] # 获取 System Prompt
29             return create_agent(model=llm, tools=tools, system_prompt=prompt)
30
31     def build_agents(model: str, api_key: str, config: AppConfig) -> AgentBundle:
32         """构建所有 Agent，每个 Agent 有不同的温度和角色"""
33         rag_config = RAGConfig(
34             milvus_host=config.milvus_host,
```

```
35         collection_name=config.milvus_collection,
36     )
37     init_rag_system(api_key=api_key, config=rag_config)
38
39     return AgentBundle(
40         intent_router=build_agent(model, api_key, "intent_router", 0.0, []),
41         planner=build_agent(model, api_key, "plan", 0.3, []),
42         scout_web=build_agent(model, api_key, "web_search", 0.4, []),
43         scout_local=build_agent(model, api_key, "local_rag", 0.4, []),
44         evidence_judge=build_agent(model, api_key, "deep_dive", 0.2, []),
45         analyst=build_agent(model, api_key, "analyze", 0.3, []),
46         direct_responder=build_agent(model, api_key, "direct_answer", 0.2, []),
47         writer=build_agent(model, api_key, "write", 0.4, []),
48     )
```

温度参数设计原则：

Agent	Temperature	设计理由
intent_router	0.0	意图判断需要确定性，不能随机
planner	0.3	规划需要一定创意，但不能太发散
scout_web/scout_local	0.4	检索需要平衡覆盖率和相关性
evidence_judge	0.2	证据评分需要严格标准
analyst	0.3	分析需要逻辑严谨
writer	0.4	写作需要一定文采和流畅度

3.2 Intent Router（意图路由器）

3.2.1 职责说明

核心任务：判断用户问题应该直接回答，还是走多 Agent 深度研究流程。

判断标准：

- **Direct（直接回答）：** 问候、自我介绍、简单事实问答、闲聊

- **Multi-Agent（深度研究）**：需要检索、多源验证、分析对比、生成报告

3.2.2 完整 Prompt

代码块

```
1  "intent_router": """你是 IntentRouter，负责把用户问题路由到 direct 或 multiagent。
2
3  【输出格式】
4  你必须只输出 JSON，格式固定为：
5  {"route":"direct|multiagent","reason":"..."}
6
7  【判断标准】
8
9  1) Direct 场景（直接回答）：
10     - 问候语："你好"、"在吗"、"早上好"
11     - 自我介绍："你是谁"、"你能做什么"
12     - 简单问答："今天星期几"、"1+1等于几"
13     - 闲聊："讲个笑话"、"天气怎么样"（未指定城市）
14
15  2) Multi-Agent 场景（深度研究）：
16     - 包含调研类词汇："调查"、"调研"、"研究"、"盘点"、"分析"
17     - 需要多源验证："来源"、"证据"、"溯源"、"验证"
18     - 需要对比："对比"、"比较"、"vs"、"哪个更好"
19     - 需要报告输出："报告"、"总结"、"趋势"、"榜单"
20     - 时间+趋势："2026年趋势"、"最新发展"、"热门项目"
21     - 专业领域查询："架构设计"、"方案"、"实现"、"落地"
22
23  【示例】
24  输入："你好"
25  输出：{"route":"direct","reason":"问候语，不需要检索"}
26
27  输入："帮我调查2026年最好的AI产品"
28  输出：{"route":"multiagent","reason":"需要调研和盘点，涉及多源检索"}
29
30  输入："Python和Java哪个更适合做AI"
31  输出：{"route":"multiagent","reason":"需要对比分析和技术调研"}"""
```

3.2.3 节点实现代码

代码块

```
1  def detect_intent(query: str) -> str:
2      """
3      规则引擎初判意图
4      作为 LLM 的辅助，提供先验判断
5      """
```

```
6 normalized_query = query.strip()
7
8 # 强制多 Agent 关键词 (出现这些词直接判定为 multiagent)
9 force_multiagent_keywords = [
10     "调查", "调研", "来源", "证据", "检索统计", "来源清单",
11     "重大新闻", "热门项目", "趋势", "新闻", "最新", "盘点",
12 ]
13
14 # 年份 + 趋势类词汇 = 多 Agent
15 if re.search(r"20\d{2}年", normalized_query) and \
16     any(word in normalized_query for word in ["趋势", "新闻", "调研", "调查",
17 "盘点"]):
18     return "multiagent"
19
20 # 包含强制关键词
21 if any(word in query for word in force_multiagent_keywords):
22     return "multiagent"
23
24 # 扩展关键词列表
25 keywords = [
26     "调研", "研究", "调查", "盘点", "热门", "趋势", "榜单",
27     "分析", "方案", "架构", "设计", "对比", "报告", "代码",
28     "实现", "落地", "检索", "知识库", "证据", "来源",
29     "溯源", "资料", "手册", "验证", "数据", "模型",
30 ]
31 return "multiagent" if any(word in query for word in keywords) else
32 "direct"
33
34 def intent_node(state: ResearchState, agent, agent_name: str) -> ResearchState:
35     """意图识别节点"""
36     logger.info("%s 开始 | agent=%s", colorize("[intent]", "cyan"),
37 colorize(agent_name, "magenta"))
38
39 # 规则引擎初判
40 rule_route = detect_intent(state["query"])
41
42 # 构建 Prompt
43 prompt = (
44     f"用户问题: {state['query']}\n"
45     f"规则引擎初判: {rule_route}\n"
46     "请输出 JSON: {\"route\": \"direct|multiagent\", \"reason\": \"...\"}"
47 )
48
49 # 调用 Agent
50 payload, content, messages = _invoke_json_agent(
51     state,
52     prompt,
```

```

50         agent,
51         agent_name,
52         "intent",
53         {"route": rule_route, "reason": "rule"} # fallback
54     )
55
56     # 校验输出
57     route = str(payload.get("route", rule_route)).strip().lower()
58     if route not in {"direct", "multiagent"}:
59         route = rule_route
60
61     logger.info("%s 路由: %s", colorize("[intent]", "green"), route)
62     return {"intent": route, "draft": content, "messages": messages}

```

3.2.4 路由逻辑

代码块

```

1  # graph.py - 条件路由
2
3  def route_after_intent(state: ResearchState) -> str:
4      """根据意图选择下一个节点"""
5      if state.get("intent") == "direct":
6          return "direct_answer"
7      return "plan"
8
9  # 工作流连接
10 workflow.add_conditional_edges(
11     "intent",
12     route_after_intent,
13     {
14         "direct_answer": "direct_answer", # 直接回答
15         "plan": "plan", # 深度研究
16     },
17 )

```

3.3 Planner（规划师）

3.3.1 职责说明

核心任务：将用户的一句话需求拆解为可执行的研究计划。

输出内容：

1. **Objective（研究目标）：**一句话概括研究目的

2. **Sub Questions（子问题）**：原问题 + 2-3个扩展子问题

3. **Outline（大纲）**：报告结构，包含章节和检索词

4. **Budget（预算）**：资源限制（轮次、来源数、Token、时间）

3.3.2 完整 Prompt

代码块

```
1  "plan": ""你是 ChiefArchitect，总架构师。你只拿到用户的一句话 Query 与空白 state。
2
3  【任务说明】
4  你的任务不是直接下搜索语法，而是先做任务拆解，将问题拆解为原问题与衍生的子问题。
5
6  【输出格式】
7  你必须只输出 JSON，不要输出 markdown，不要补充解释。
8
9  JSON 结构固定为：
10 {
11   "objective": "研究目标，一句话概括",
12   "sub_questions": [
13     "核心原问题",
14     "扩展子问题1",
15     "扩展子问题2"
16   ],
17   "outline": [
18     {
19       "id": "sec_1",
20       "title": "章节标题",
21       "description": "章节描述",
22       "section_type": "mixed",
23       "requires_data": true,
24       "requires_chart": false,
25       "priority": 1,
26       "search_queries": ["检索词1", "检索词2"],
27       "status": "pending"
28     }
29   ],
30   "research_questions": [
31     "研究问题1",
32     "研究问题2"
33   ],
34   "budget": {
35     "max_rounds": 2,
36     "max_sources": 12,
37     "max_tokens": 12000,
38     "max_seconds": 45
```

```
39     }
40 }
41
42 【字段说明】
43 - objective: 研究目标，简洁明了
44 - sub_questions: 必须包含1个核心原问题和2-3个扩展子问题
45 - outline: 报告大纲，每个章节包含检索词(search_queries)
46 - search_queries: 必须是针对子问题的自然语言检索词，不是关键词堆砌
47 - budget: 资源预算，控制研究范围
48
49 【示例】
50 输入: "帮我调查2026年最好的AI产品"
51
52 输出:
53 {
54     "objective": "调查2026年最佳AI产品，覆盖各细分领域",
55     "sub_questions": [
56         "2026年有哪些值得关注的AI产品",
57         "各细分领域（写作、绘画、编程）的最佳产品是什么",
58         "这些产品的用户评价和专业评测如何"
59     ],
60     "outline": [
61         {
62             "id": "sec_1",
63             "title": "2026年AI产品市场概况",
64             "description": "整体市场趋势和热门方向",
65             "section_type": "mixed",
66             "requires_data": true,
67             "requires_chart": false,
68             "priority": 1,
69             "search_queries": [
70                 "2026年AI产品发展趋势",
71                 "2026 best AI products market"
72             ],
73             "status": "pending"
74         },
75         {
76             "id": "sec_2",
77             "title": "各细分领域最佳产品",
78             "description": "写作、绘画、编程等领域的头部产品",
79             "section_type": "mixed",
80             "requires_data": true,
81             "requires_chart": false,
82             "priority": 2,
83             "search_queries": [
84                 "2026 best AI writing tools",
85                 "2026 best AI image generation",
```

```

86         "2026 best AI coding assistants"
87     ],
88     "status": "pending"
89 }
90 ],
91 "research_questions": [
92     "2026年AI产品市场整体趋势如何",
93     "各细分领域有哪些领先产品",
94     "用户对这些产品的真实评价如何"
95 ],
96 "budget": {
97     "max_rounds": 2,
98     "max_sources": 12,
99     "max_tokens": 12000,
100    "max_seconds": 45
101 }
102 }"""

```

3.3.3 节点实现代码

代码块

```

1  def _default_plan(state: ResearchState) -> dict:
2      """默认回退方案，当 LLM 输出解析失败时使用"""
3      return {
4          "objective": state["query"],
5          "sub_questions": [state["query"]],
6          "outline": [
7              {
8                  "id": "sec_1",
9                  "title": "默认大纲",
10                 "description": "默认生成的大纲",
11                 "section_type": "mixed",
12                 "requires_data": False,
13                 "requires_chart": False,
14                 "priority": 1,
15                 "search_queries": [state["query"]],
16                 "status": "pending",
17             }
18         ],
19         "research_questions": [state["query"]],
20         "budget": {"max_rounds": 2, "max_sources": 12, "max_tokens": 12000,
21                    "max_seconds": 45},
22     }
23  def _derive_search_plan(outline: list[dict], sub_questions: list[str],

```



```

24             research_questions: list[str], query: str) ->
list[dict]:
25     """从大纲派生搜索计划"""
26     plan: list[dict] = []
27
28     # 1. 添加直接搜索查询 (围绕用户原始问题)
29     for direct_query in _derive_direct_search_queries(query):
30         plan.append({
31             "section_id": "user_query",
32             "query": direct_query,
33             "source_preference": "hybrid",
34             "reason": "围绕用户原始问题生成的直接检索词",
35         })
36
37     # 2. 从大纲章节提取搜索查询
38     for section in outline:
39         if not isinstance(section, dict):
40             continue
41         section_id = str(section.get("id") or "sec")
42         for item in section.get("search_queries", []) or []:
43             text = str(item).strip()
44             if text and _is_query_grounded(text, query):
45                 plan.append({
46                     "section_id": section_id,
47                     "query": text,
48                     "source_preference": "hybrid",
49                     "reason": f"来自大纲章节 {section_id}",
50                 })
51
52     # 3. 去重
53     if not plan:
54         plan.append({"section_id": "sec_1", "query": query,
55 "source_preference": "hybrid", "reason": "fallback"})
56
57     deduped = _dedupe_sources(plan, ["query"])
58     return deduped[:6] # 最多6个查询
59
60 def plan_node(state: ResearchState, agent, agent_name: str) -> ResearchState:
61     """规划节点"""
62     logger.info("%s 开始 | agent=%s", colorize("[plan]", "cyan"),
63 colorize(agent_name, "magenta"))
64     log_inputs("plan", agent_name, {"query": state["query"]})
65
66     fallback = _default_plan(state)
67
68     # 调用 Agent
69     payload, content, messages = _invoke_json_agent(

```

```

68         state,
69         f"用户需求: {state['query']}\n请先做大纲与问题拆解, 再输出规划 JSON。 ",
70         agent,
71         agent_name,
72         "plan",
73         fallback,
74     )
75
76     # 提取规划结果 (带类型检查)
77     outline = payload.get("outline") if isinstance(payload.get("outline"),
78 list) else fallback["outline"]
79     sub_questions = payload.get("sub_questions") if
80 isinstance(payload.get("sub_questions"), list) else fallback["sub_questions"]
81     research_questions = payload.get("research_questions") if
82 isinstance(payload.get("research_questions"), list) else
83 fallback["research_questions"]
84     budget = payload.get("budget") if isinstance(payload.get("budget"), dict)
85 else fallback["budget"]
86
87     # 派生搜索计划
88     search_plan = _derive_search_plan(outline, sub_questions,
89 research_questions, state["query"])
90     plan_summary = payload.get("objective") or state["query"]
91
92     return {
93         "phase": "planning completed",
94         "plan": plan_summary,
95         "outline": outline,
96         "sub_questions": sub_questions,
97         "research_questions": research_questions,
98         "search_plan": search_plan,
99         "budget": budget,
100        "messages": messages,
101        "draft": content,
102        "iteration": 0,
103    }

```

3.4 Web Scout (网络侦察员)

3.4.1 职责说明

核心任务:

1. 执行网络搜索 (Bocha AI Search API)

2. 过滤相关性低的搜索结果
3. 结构化整理证据，分配 source_id
4. 识别信息缺口

source_id 命名规则：`WEB{迭代}\_{查询序号}-{结果序号}`

- 示例：WEB1_1-1 表示第1轮迭代，第1个查询的第1条结果

3.4.2 完整 Prompt

代码块

```
1  "web_search": ""你是 WebScout，负责网络取证与相关性过滤。
2
3  【输入内容】
4  你会拿到：
5  1. 用户原问题
6  2. 子问题列表
7  3. 网页原始证据（带 source_id、title、url、snippet、domain）
8
9  【处理任务】
10 1. 相关性判断：判断每条证据是否与"原问题或任一子问题"相关
11   - 只要包含用户问题中核心实体的有效信息或线索，就予以保留
12   - 明显无关或广告的则丢弃
13   - 如果无法判断相关性但包含问题字眼，请倾向于保留
14
15 2. 可信度标记：
16   - official：官方网站（.gov、.edu、官方域名）
17   - media：主流媒体（新闻网站、知名科技媒体）
18   - community：社区/论坛（知乎、Reddit、GitHub Issues）
19   - unknown：无法判断
20
21 3. 关联子问题：每条证据应该支持哪些子问题
22
23 【输出格式】
24 你必须只输出 JSON，不要输出 markdown，不要补充解释。
25
26 JSON 结构固定为：
27 {
28   "summary": "证据采集总结，2-3句话概括",
29   "evidence": [
30     {
31       "source_id": "WEB1_1-1",
32       "title": "网页标题",
33       "url": "https://...",
34       "snippet": "内容摘要",
35       "domain": "example.com",
```

```

36     "source_type": "web",
37     "reliability_hint": "official|media|community|unknown",
38     "supports_questions": ["子问题1", "子问题2"],
39     "notes": "补充说明（可选）"
40 }
41 ],
42 "gaps": ["信息缺口1", "信息缺口2"],
43 "rejected_source_ids": ["WEB1_1-3", "WEB1_2-1"],
44 "reject_reason": "被拒绝的证据主要是因为..."
45 }
46
47 【重要约束】
48 - evidence 里只能出现输入里存在的 source_id，不能编造
49 - 如果无法判断相关性但包含问题字眼，请倾向于保留
50 - 确属无关的放入 rejected_source_ids，并在 reject_reason 说明原因
51 - 不要输出任何解释性文字，只输出 JSON"""

```

3.4.3 节点实现代码

代码块

```

1  def _build_queries(state: ResearchState, source_preference: str) -> list[dict]:
2      """构建查询列表"""
3      queries: list[dict] = []
4
5      # 判断是否在补搜迭代中
6      iteration = state.get("iteration", 0)
7      if iteration > 0 and state.get("supplementary_queries"):
8          base_plan = state.get("supplementary_queries", [])
9      else:
10         base_plan = state.get("search_plan", [])
11
12     # 筛选符合 source_preference 的查询
13     for item in base_plan:
14         if not isinstance(item, dict):
15             continue
16         pref = item.get("source_preference", "hybrid")
17         if pref in (source_preference, "hybrid"):
18             query = str(item.get("query", "")).strip()
19             if query:
20                 queries.append(item)
21
22     if not queries:
23         queries.append({"section_id": "sec_1", "query": state["query"],
24                         "source_preference": source_preference, "reason":
25                         "fallback"})

```

```
25     return queries[:6]
26
27 def _assign_source_ids(records: list[dict], prefix: str) -> list[dict]:
28     """为记录分配 source_id"""
29     assigned: list[dict] = []
30     for index, record in enumerate(records, 1):
31         item = dict(record)
32         item["source_id"] = f"{prefix}-{index}"
33         assigned.append(item)
34     return assigned
35
36 def _filter_web_records(query: str, records: list[dict]) -> tuple[list[dict],
dict]:
37     """过滤网页记录"""
38     kept = []
39     stats = {"raw_count": len(records), "kept_count": 0,
40             "dropped_irrelevant": 0, "dropped_domain": 0, "dropped_empty": 0}
41
42     for record in records:
43         title = str(record.get("title", ""))
44         snippet = str(record.get("snippet", ""))
45         domain = str(record.get("domain", ""))
46
47         # 过滤空记录
48         if not title and not snippet:
49             stats["dropped_empty"] += 1
50             continue
51
52         # 过滤黑名单域名
53         if _is_bad_web_domain(domain):
54             stats["dropped_domain"] += 1
55             continue
56
57         # 计算相关性
58         relevance = _estimate_relevance(query, f"{title}\n{snippet}")
59         record["relevance_score"] = relevance
60
61         # 过滤低相关性（非官方域名）
62         if relevance < 0.2 and not _is_official_domain(domain):
63             stats["dropped_irrelevant"] += 1
64             continue
65
66         kept.append(record)
67
68     stats["kept_count"] = len(kept)
69     return kept, stats
70
```

```

71 def web_search_node(state: ResearchState, agent, agent_name: str) ->
    ResearchState:
72     """网络搜索节点"""
73     logger.info("%s 开始 | agent=%s", colorize("[web_search]", "cyan"),
        colorize(agent_name, "magenta"))
74
75     # 1. 构建查询
76     queries = _build_queries(state, "web")
77     raw_records = []
78     query_traces = state.get("web_search_trace", [])
79
80     iteration = state.get("iteration", 0)
81     prefix = f"WEB{iteration+1}"
82
83     # 2. 执行搜索
84     for query_index, item in enumerate(queries, 1):
85         # 调用 Bocha API, count=4 减少无用 Token
86         records = bocha_web_search_records(str(item.get("query", "")), count=4)
87         records = _assign_source_ids(records, f"{prefix}_{query_index}")
88
89         for record in records:
90             record["section_id"] = item.get("section_id")
91             record["search_query"] = item.get("query")
92
93         raw_records.extend(records)
94         query_traces.append({
95             "iteration": iteration,
96             "plan_step": query_index,
97             "query": str(item.get("query", "")),
98             "section_id": item.get("section_id"),
99             "reason": item.get("reason", ""),
100             "source_preference": item.get("source_preference", "web"),
101             "raw_count": len(records),
102             "raw_records": _summarize_records(records),
103         })
104
105     # 3. 去重和过滤
106     raw_records = _dedupe_sources(raw_records, ["url", "title"])
107     raw_records = _minimal_record_filter(raw_records, ["title", "snippet",
        "url"])
108
109     # 4. 统计
110     web_retrieval_stats = state.get("web_retrieval_stats", {})
111     web_retrieval_stats["query_count"] =
web_retrieval_stats.get("query_count", 0) + len(queries)
112     web_retrieval_stats["raw_count"] = web_retrieval_stats.get("raw_count", 0)
        + len(raw_records)

```

```
113
114     log_inputs("web_search", agent_name, {"query_count": str(len(queries)),
"raw_count": str(len(raw_records))})
115
116     # 5. 无结果处理
117     if not raw_records:
118         logger.info("%s 无可可用网页证据，跳过网页上下文注入", colorize("
[web_search]", "yellow"))
119         return {
120             "web_search": "未检索到可用网页证据，已跳过网页上下文注入。",
121             "web_evidence": state.get("web_evidence", []),
122             "web_retrieval_stats": web_retrieval_stats,
123             "web_search_trace": query_traces,
124         }
125
126     # 6. LLM 结构化处理
127     fallback = _fallback_web_evidence(raw_records)
128     payload, content, messages = _invoke_json_agent(
129         state,
130         "请基于以下网页证据整理结构化 JSON。 \n"
131         f"原问题: {state['query']}\n"
132         f"子问题: {json.dumps(state.get('sub_questions', []),
ensure_ascii=False)}\n"
133         f"原始网页证据: \n{_format_raw_records(raw_records, 'web')}",
134         agent,
135         agent_name,
136         "web_search",
137         fallback,
138     )
139
140     # 7. 校验和补充
141     evidence = payload.get("evidence") if isinstance(payload.get("evidence"),
list) else fallback["evidence"]
142     allowed_source_ids = {str(item.get("source_id")) for item in raw_records if
item.get("source_id")}
143     evidence = _prune_evidence_to_allowed_sources(evidence, allowed_source_ids)
144     evidence = _enrich_evidence_from_raw(evidence, raw_records) # 补充 URL
145
146     # 8. 更新统计
147     web_retrieval_stats["kept_count"] = web_retrieval_stats.get("kept_count",
0) + len(evidence)
148     web_retrieval_stats["dropped_count"] =
web_retrieval_stats.get("dropped_count", 0) + max(len(raw_records) -
len(evidence), 0)
149
150     kept_ids = {str(item.get("source_id")) for item in evidence if
item.get("source_id")}
```

```
151     query_traces = _finalize_query_traces(query_traces, kept_ids,
152                                           payload.get("rejected_source_ids",
153               []),
154                                           str(payload.get("reject_reason",
155               ""))).strip())
156
157     existing_evidence = state.get("web_evidence", [])
158     return {
159         "web_search": payload.get("summary", content),
160         "web_evidence": existing_evidence + evidence,
161         "web_retrieval_stats": web_retrieval_stats,
162         "web_search_trace": query_traces,
163         "messages": messages,
164     }
```

3.5 Local RAG Scout（本地知识侦察员）

3.5.1 职责说明

核心任务：

- 1. 检索本地 Milvus 向量数据库
- 2. 对检索结果进行相关性过滤
- 3. 结构化整理证据，分配 source_id
- 4. 与 Web Scout 并行执行

source_id 命名规则：`LOC{迭代}\_{查询序号}-{结果序号}`

- 示例：LOC1_1-1 表示第1轮迭代，第1个查询的第1条结果

与 Web Scout 的区别：

维度	Web Scout	Local RAG Scout
数据来源	Bocha 网络搜索 API	Milvus 向量数据库
数据类型	网页、新闻、博客	企业内部文档、知识库
可信度	需要评估（0.5-0.9）	默认高可信（0.92）
定位器	URL	doc_id（文档路径）

3.5.2 完整 Prompt

代码块local_rag": """"你是 LocalRAGScout, 负责本地知识库取证与相关性过滤。

2

3 **【输入内容】**

4 你会拿到:

- 5 1. 用户原问题
- 6 2. 子问题列表
- 7 3. 知识库检索原始结果 (带 source_id、doc_id、title、snippet)

8

9 **【处理任务】**

- 10 1. 相关性判断: 判断每条证据是否与"原问题或任一子问题"相关
 - 11 - 本地知识库内容默认可信度高, 但仍需判断相关性
 - 12 - 只要包含用户问题中核心实体的有效信息, 就予以保留
 - 13 - 明显无关的则丢弃
- 14
- 15 2. 可信度标记:
 - 16 - internal: 企业内部知识库 (默认)
- 17
- 18 3. 关联子问题: 每条证据应该支持哪些子问题

19

20 **【输出格式】**

21 你必须只输出 JSON, 不要输出 markdown, 不要补充解释。

22

23 JSON 结构固定为:

```
24   {
25     "summary": "本地知识库证据采集总结",
26     "evidence": [
27       {
28         "source_id": "LOC1_1-1",
29         "doc_id": "/docs/ai_guide.pdf",
30         "title": "AI产品选型指南",
31         "snippet": "内容摘要",
32         "source_type": "local",
33         "reliability_hint": "internal",
34         "supports_questions": ["子问题1"],
35         "notes": "补充说明 (可选) "
36       }
37     ],
38     "gaps": ["本地知识库未覆盖的缺口"],
39     "rejected_source_ids": ["LOC1_1-2"],
40     "reject_reason": "被拒绝的证据主要是因为..."
41   }
```

42

43 **【重要约束】**

- 44 - evidence 里只能出现输入里存在的 source_id, 不能虚构文档
- 45 - 本地知识库证据默认高可信, 但仍需判断相关性
- 46 - 确属无关的放入 rejected_source_ids
- 47 - 不要输出任何解释性文字, 只输出 JSON"""

3.5.3 节点实现代码

代码块

```
1  def local_rag_node(state: ResearchState, agent, agent_name: str) ->
    ResearchState:
2      """本地 RAG 节点"""
3      logger.info("%s 开始 | agent=%s", colorize("[local_rag]", "cyan"),
        colorize(agent_name, "magenta"))
4
5      # 1. 构建查询 (复用 Web Scout 的查询计划)
6      queries = _build_queries(state, "local")
7      raw_records = []
8      query_traces = state.get("local_rag_trace", [])
9
10     iteration = state.get("iteration", 0)
11     prefix = f"LOC{iteration+1}"
12
13     # 2. 执行本地检索
14     for query_index, item in enumerate(queries, 1):
15         # 查询 Milvus 向量数据库
16         records = search_knowledge_base_records(str(item.get("query", "")),
            limit=4)
17         records = _assign_source_ids(records, f"{prefix}_{query_index}")
18
19         for record in records:
20             record["section_id"] = item.get("section_id")
21             record["search_query"] = item.get("query")
22
23         raw_records.extend(records)
24         query_traces.append({
25             "iteration": iteration,
26             "plan_step": query_index,
27             "query": str(item.get("query", "")),
28             "section_id": item.get("section_id"),
29             "reason": item.get("reason", ""),
30             "source_preference": item.get("source_preference", "local"),
31             "raw_count": len(records),
32             "raw_records": _summarize_records(records),
33         })
34
35     # 3. 去重和过滤
36     raw_records = _dedupe_sources(raw_records, ["doc_id", "snippet"])
37     raw_records = _minimal_record_filter(raw_records, ["snippet", "title",
        "doc_id"])
38
```

```

39     # 4. 统计
40     local_retrieval_stats = state.get("local_retrieval_stats", {})
41     local_retrieval_stats["query_count"] =
local_retrieval_stats.get("query_count", 0) + len(queries)
42     local_retrieval_stats["raw_count"] =
local_retrieval_stats.get("raw_count", 0) + len(raw_records)
43
44     log_inputs("local_rag", agent_name, {"query_count": str(len(queries)),
"raw_count": str(len(raw_records))})
45
46     # 5. 无结果处理
47     if not raw_records:
48         logger.info("%s 无可本地证据, 跳过本地上下文注入", colorize("
[local_rag]", "yellow"))
49         return {
50             "local_rag": "未检索到可用本地知识库证据, 已跳过本地上下文注入。",
51             "local_evidence": state.get("local_evidence", []),
52             "local_retrieval_stats": local_retrieval_stats,
53             "local_rag_trace": query_traces,
54         }
55
56     # 6. LLM 结构化处理
57     fallback = _fallback_local_evidence(raw_records)
58     payload, content, messages = _invoke_json_agent(
59         state,
60         "请基于以下知识库证据整理结构化 JSON。 \n"
61         f"原问题: {state['query']}\n"
62         f"子问题: {json.dumps(state.get('sub_questions', []))},
ensure_ascii=False)\n"
63         f"原始知识库证据: \n{_format_raw_records(raw_records, 'local')}",
64         agent,
65         agent_name,
66         "local_rag",
67         fallback,
68     )
69
70     # 7. 校验
71     evidence = payload.get("evidence") if isinstance(payload.get("evidence"),
list) else fallback["evidence"]
72     allowed_source_ids = {str(item.get("source_id")) for item in raw_records if
item.get("source_id")}
73     evidence = _prune_evidence_to_allowed_sources(evidence, allowed_source_ids)
74
75     # 8. 更新统计
76     local_retrieval_stats["kept_count"] =
local_retrieval_stats.get("kept_count", 0) + len(evidence)

```

```
77     local_retrieval_stats["dropped_count"] =
    local_retrieval_stats.get("dropped_count", 0) + max(len(raw_records) -
    len(evidence), 0)
78
79     kept_ids = {str(item.get("source_id")) for item in evidence if
    item.get("source_id")}
80     query_traces = _finalize_query_traces(query_traces, kept_ids,
81                                           payload.get("rejected_source_ids",
    []),
82                                           str(payload.get("reject_reason",
    ""))).strip())
83
84     existing_evidence = state.get("local_evidence", [])
85     return {
86         "local_rag": payload.get("summary", content),
87         "local_evidence": existing_evidence + evidence,
88         "local_retrieval_stats": local_retrieval_stats,
89         "local_rag_trace": query_traces,
90         "messages": messages,
91     }
```

3.6 Evidence Judge（证据裁判）

3.6.1 职责说明

核心任务：

- 1. 评分：对每条证据进行可信度评分（0-1）
- 2. 去重：基于 URL/doc_id 去重
- 3. 冲突检测：识别证据之间的矛盾
- 4. 来源索引：构建统一的 source_index

评分标准：

来源类型	分数	说明
本地知识库	0.92	企业内部数据，默认高可信
官方/政府域名	0.88	.gov、.edu、官方站点
主流媒体	0.72	新闻网站、知名科技媒体
普通网站	0.58	需要交叉验证

3.6.2 完整 Prompt

代码块

```
1  "deep_dive": """"你是 EvidenceJudge，负责证据裁判。
2
3  【输入内容】
4  你会拿到：
5  1. web_evidence：网络检索证据列表
6  2. local_evidence：本地知识库证据列表
7  3. sub_questions：子问题列表
8
9  【处理任务】
10 1. 证据评分 (reliability_score)：
11   - 本地知识库：0.92（企业内部数据）
12   - 官方/政府域名：0.88（.gov、.edu）
13   - 主流媒体：0.72（新闻、知名科技媒体）
14   - 普通网站：0.58（需要交叉验证）
15   - 来源不明：0.45（信息不完整）
16
17 2. 去重：
18   - 相同 URL 或 doc_id 的证据只保留一条
19   - 保留内容最完整的一条
20
21 3. 冲突检测 (audit_flags)：
22   - low_confidence：可信度低于 0.6 的证据
23   - conflict：与其他证据矛盾的信息
24   - missing_evidence：缺少直接证据支持的子问题
25
26 4. 构建来源索引 (source_index)：
27   - 为每条证据生成 source_id、label、locator
28
29 【输出格式】
30 你必须只输出 JSON，不要输出 markdown。
31
32 JSON 结构固定为：
33 {
34   "summary": "证据裁判总结",
35   "evidence_pool": [
36     {
37       "source_id": "WEB1_1-1",
38       "source_type": "web|local",
39       "title": "证据标题",
40       "url": "https://...",
41       "doc_id": "...",
```

```

42     "snippet": "内容摘要",
43     "supports_questions": ["子问题1"],
44     "reliability_score": 0.82,
45     "reliability_reason": "主流媒体域名",
46     "source_label": "显示名称"
47 }
48 ],
49 "audit_flags": [
50     {
51         "type": "low_confidence|conflict|missing_evidence",
52         "target": "问题1",
53         "reason": "说明原因"
54     }
55 ],
56 "source_index": [
57     {
58         "source_id": "WEB1_1-1",
59         "label": "显示名称",
60         "locator": "URL或doc_id",
61         "source_type": "web|local"
62     }
63 ]
64 }
65
66 【重要约束】
67 - 本地知识库和官方站点优先高分
68 - 自媒体和论坛低分
69 - 冲突必须显式标记
70 - 不要输出任何解释性文字，只输出 JSON"""

```

3.6.3 节点实现代码

代码块

```

1  def _score_evidence(record: dict) -> tuple[float, str]:
2      """证据评分算法"""
3      source_type = record.get("source_type")
4
5      # 本地知识库最高可信
6      if source_type == "local":
7          return 0.92, "企业内部知识库证据，默认高可信"
8
9      domain = str(record.get("domain", "")).lower()
10
11     # 官方/政府域名
12     if _is_official_domain(domain):

```

```
13         return 0.88, "官方或权威机构域名"
14
15     # 主流媒体
16     if any(word in domain for word in ["news", "finance", "reuters",
17 "bloomberg", "people", "xinhuanet"]):
18         return 0.72, "主流媒体域名"
19
20     # 普通网站
21     if domain:
22         return 0.58, "普通互联网来源, 需要交叉验证"
23
24     # 来源不明
25     return 0.45, "来源信息不完整"
26
27 def _is_official_domain(domain: str) -> bool:
28     """判断是否为官方域名"""
29     value = domain.lower()
30     return (value.endswith(".gov.cn") or value.endswith(".gov") or
31             value.endswith(".edu") or value.endswith(".edu.cn") or
32             "gov" in value or "official" in value)
33
34 def deep_dive_node(state: ResearchState, agent, agent_name: str) ->
35 ResearchState:
36     """深度分析/证据裁判节点"""
37     logger.info("%s 开始 | agent=%s", colorize("[deep_dive]", "cyan"),
38 colorize(agent_name, "magenta"))
39
40     # 1. 检查是否有证据
41     if not state.get("web_evidence") and not state.get("local_evidence"):
42         logger.info("%s 等待检索结果", colorize("[deep_dive]", "yellow"))
43         return {}
44
45     # 2. 回退方案
46     fallback = _fallback_audit(state)
47
48     # 3. 调用 Agent
49     payload, content, messages = _invoke_json_agent(
50         state,
51         "请对 web 与 local 证据进行评分、去重、冲突审计, 并只输出 JSON。 \n"
52         f"问题: {state['query']}\n"
53         f"子问题: {json.dumps(state.get('sub_questions', []),
54 ensure_ascii=False)}\n"
55         f"web_evidence: {json.dumps(state.get('web_evidence', []),
56 ensure_ascii=False)}\n"
57         f"local_evidence: {json.dumps(state.get('local_evidence', []),
58 ensure_ascii=False)}",
59         agent,
```

```

54         agent_name,
55         "deep_dive",
56         fallback,
57     )
58
59     # 4. 处理证据池
60     payload_pool = payload.get("evidence_pool") if
isinstance(payload.get("evidence_pool"), list) else []
61     raw_evidence = state.get("web_evidence", []) + state.get("local_evidence",
[])
62
63     # 5. 校验 source_id 合法性
64     allowed_source_ids = {str(item.get("source_id", "")).strip() for item in
raw_evidence if item.get("source_id")}
65     evidence_pool = []
66     for item in payload_pool:
67         if not isinstance(item, dict):
68             continue
69         sid = str(item.get("source_id", "")).strip()
70         if sid and sid in allowed_source_ids:
71             evidence_pool.append(item)
72
73     # 6. 补充缺失的证据 (LLM 可能漏掉)
74     if not evidence_pool:
75         evidence_pool = fallback["evidence_pool"]
76
77     existing_ids = {str(item.get("source_id", "")).strip() for item in
evidence_pool if isinstance(item, dict)}
78     for record in raw_evidence:
79         sid = str(record.get("source_id", "")).strip()
80         if not sid or sid in existing_ids:
81             continue
82         score, reason = _score_evidence(record)
83         evidence_pool.append({
84             "source_id": sid,
85             "source_type": record.get("source_type", "source"),
86             "title": record.get("title") or sid,
87             "url": record.get("url", ""),
88             "doc_id": record.get("doc_id", ""),
89             "snippet": record.get("snippet", ""),
90             "supports_questions": record.get("supports_questions", []),
91             "reliability_score": score,
92             "reliability_reason": reason,
93             "source_label": record.get("title") or record.get("doc_id") or
record.get("url") or sid,
94         })
95     existing_ids.add(sid)

```



```

96
97     # 7. 构建来源索引
98     audit_flags = payload.get("audit_flags") if
isinstance(payload.get("audit_flags"), list) else fallback["audit_flags"]
99     source_index = []
100     for item in evidence_pool:
101         if not isinstance(item, dict):
102             continue
103         sid = str(item.get("source_id", "")).strip()
104         if not sid:
105             continue
106         source_index.append({
107             "source_id": sid,
108             "label": item.get("title") or item.get("source_label") or sid,
109             "locator": item.get("url") or item.get("doc_id") or "",
110             "source_type": item.get("source_type", "source"),
111         })
112     source_index = _dedupe_sources(source_index, ["source_id"])
113
114     return {
115         "deep_dive": payload.get("summary", content),
116         "audit": payload.get("summary", content),
117         "evidence_pool": evidence_pool,
118         "audit_flags": audit_flags,
119         "source_index": source_index,
120         "messages": messages,
121     }

```

3.7 Analyst（分析师）

3.7.1 职责说明

核心任务：

1. 从证据池中形成结论（findings）
2. 建立结论-来源映射（claim_map）
3. 评估证据完备性
4. 识别信息缺口（missing_gaps）
5. 决定是否需要补搜（needs_more_research）

决策逻辑：

代码块

```
1  证据充足 —▶ needs_more_research=false —▶ 进入 Write
2      |
3  证据不足 —▶ needs_more_research=true —▶ 进入 Reflect
4      |
5  达到 max_iterations —▶ 强制进入 Write
```

3.7.2 完整 Prompt

代码块

```
1  "analyze": ""你是 Analyst，负责从证据池中形成结论并评估证据完备性。
2
3  【输入内容】
4  你会拿到：
5  1. 用户原问题
6  2. 子问题列表
7  3. 证据池 (evidence_pool)
8  4. 审计标记 (audit_flags)
9
10 【处理任务】
11 1. 形成结论 (findings)：
12   - 基于证据池中的信息，形成对子问题的回答
13   - 每个结论必须有明确的 claim (论断)
14   - 每个结论必须有 confidence (high/medium/low)
15   - 每个结论必须绑定 source_ids (来源证据)
16
17 2. 结论-来源映射 (claim_map)：
18   - 建立结论 ID 到来源 ID 列表的映射
19
20 3. 评估证据完备性：
21   - 检查每个子问题是否有足够的证据支持
22   - 如果不足，设置 needs_more_research=true
23   - 列出 missing_gaps (信息缺口)
24
25 4. 下一步行动 (next_actions)：
26   - 如果证据充足：["撰写最终报告"]
27   - 如果证据不足：["补充关于XX的检索", "补充关于YY的检索"]
28
29 【输出格式】
30 你必须只输出 JSON，不要输出 markdown。
31
32 JSON 结构固定为：
33 {
34   "analysis_summary": "分析总结，2-3句话",
```

```

35     "needs_more_research": false,
36     "missing_gaps": ["信息缺口1", "信息缺口2"],
37     "findings": [
38         {
39             "claim_id": "c_1",
40             "claim": "结论论断",
41             "confidence": "high|medium|low",
42             "source_ids": ["WEB1_1-1", "LOC1_1-2"]
43         }
44     ],
45     "claim_map": [
46         {
47             "claim_id": "c_1",
48             "source_ids": ["WEB1_1-1", "LOC1_1-2"]
49         }
50     ],
51     "next_actions": ["下一步行动"]
52 }
53
54 【重要约束】
55 - 每个结论必须绑定来源 source_id
56 - 证据不足时明确写 uncertain
57 - needs_more_research 必须诚实评估
58 - 不要输出任何解释性文字，只输出 JSON"""

```

3.7.3 节点实现代码

代码块

```

1  def _fallback_analysis(state: ResearchState) -> dict:
2      """分析回退方案"""
3      source_ids = [item.get("source_id") for item in state.get("evidence_pool",
4          [])[:3] if item.get("source_id")]
5      return {
6          "analysis_summary": "默认分析结论",
7          "needs_more_research": False,
8          "missing_gaps": [],
9          "findings": [
10             {
11                 "claim_id": "c_1",
12                 "claim": f"围绕「{state['query']}」已完成多源检索，初步证据表明问题
13                 可以从网络与本地知识库双侧支撑。",
14                 "confidence": "medium" if source_ids else "low",
15                 "source_ids": source_ids,
16             }
17         ],

```

```
16         "claim_map": [{"claim_id": "c_1", "source_ids": source_ids}],
17         "next_actions": [] if source_ids else ["补充更多高质量来源"],
18     }
19
20 def analyze_node(state: ResearchState, agent, agent_name: str) ->
ResearchState:
21     """分析节点"""
22     logger.info("%s 开始 | agent=%s", colorize("[analyze]", "cyan"),
colorize(agent_name, "magenta"))
23
24     fallback = _fallback_analysis(state)
25
26     # 调用 Agent
27     payload, content, messages = _invoke_json_agent(
28         state,
29         "请基于证据池输出结论映射 JSON，并评估证据完备性：\n"
30         f"原问题：{state['query']}\n"
31         f"子问题：{json.dumps(state.get('sub_questions', []),
ensure_ascii=False)}\n"
32         f"证据池：{json.dumps(state.get('evidence_pool', []),
ensure_ascii=False)}\n"
33         f"审计标记：{json.dumps(state.get('audit_flags', []),
ensure_ascii=False)}",
34         agent,
35         agent_name,
36         "analyze",
37         fallback,
38     )
39
40     # 提取结果
41     findings = payload.get("findings") if isinstance(payload.get("findings"),
list) else fallback["findings"]
42     claim_map = payload.get("claim_map") if
isinstance(payload.get("claim_map"), list) else fallback["claim_map"]
43     needs_more_research = payload.get("needs_more_research", False)
44     missing_gaps = payload.get("missing_gaps", [])
45     analysis_summary = payload.get("analysis_summary", content)
46
47     return {
48         "analysis": analysis_summary,
49         "findings": findings,
50         "claim_map": claim_map,
51         "needs_more_research": needs_more_research,
52         "missing_gaps": missing_gaps,
53         "messages": messages,
54     }
```

3.8 Reflect（反思补搜）

3.8.1 职责说明

核心任务：

1. 接收分析师指出的信息缺口（missing_gaps）
2. 生成新的、更具针对性的搜索词
3. 避免重复之前已经尝试过的查询

触发条件：

- `needs_more_research=true`
- 当前迭代次数 < max_iterations

3.8.2 完整 Prompt

代码块

```
1  "reflect": ""你是 ResearchPlanner，负责基于分析师的反馈生成补搜计划。
2
3  【输入内容】
4  你会拿到：
5  1. 原问题
6  2. 子问题列表
7  3. 已执行过的搜索计划
8  4. 已执行过的补搜计划
9  5. 分析师指出的信息缺口（missing_gaps）
10
11 【处理任务】
12 1. 分析信息缺口：
13    - 理解 missing_gaps 中每个缺口的含义
14    - 确定需要补充哪些方面的信息
15
16 2. 生成补搜查询：
17    - 新的搜索词必须与之前的搜索词不同
18    - 可以尝试换词、加限定词或拆解更细的查询
19    - 针对每个缺口生成 1-2 个查询
20
21 3. 指定来源偏好：
22    - web：优先网络搜索
23    - local：优先本地知识库
24    - hybrid：两者都搜索
25
26 【输出格式】
```

```

27 你必须只输出 JSON，不要输出 markdown。
28
29 JSON 结构固定为：
30 {
31     "reflection_summary": "反思总结，说明补搜策略",
32     "supplementary_queries": [
33         {
34             "section_id": "gap_1",
35             "query": "新的搜索词",
36             "source_preference": "hybrid",
37             "reason": "为什么这个查询能填补缺口"
38         }
39     ]
40 }
41
42 【重要约束】
43 - 新的搜索词必须与之前的搜索词不同
44 - 不要重复已经尝试过的查询
45 - 针对性强，直接回应 missing_gaps
46 - 不要输出任何解释性文字，只输出 JSON"""

```

3.8.3 节点实现代码

代码块

```

1  def reflect_node(state: ResearchState, agent, agent_name: str) ->
    ResearchState:
2      """反思/补搜节点"""
3      logger.info("%s 开始 | agent=%s", colorize("[reflect]", "cyan"),
4                  colorize(agent_name, "magenta"))
5
6      missing_gaps = state.get("missing_gaps", [])
7      log_inputs("reflect", agent_name, {"missing_gaps": str(missing_gaps)})
8
9      # 回退方案
10     fallback = {
11         "reflection_summary": "默认补搜",
12         "supplementary_queries": [
13             {"section_id": "gap_1", "query": state["query"],
14             "source_preference": "hybrid", "reason": "fallback"}
15         ]
16     }
17
18     # 构建 Prompt
19     prompt = (
20         f"分析师指出当前证据不足以完全回答问题，存在以下信息缺口：\n"

```

```

19         f"{json.dumps(missing_gaps, ensure_ascii=False)}\n\n"
20         f"原问题: {state['query']}\n"
21         f"子问题: {json.dumps(state.get('sub_questions', []),
ensure_ascii=False)}\n"
22         f"已执行过的搜索计划: \n{json.dumps(state.get('search_plan', []),
ensure_ascii=False)}\n"
23         f"已执行过的补搜计划: \n{json.dumps(state.get('supplementary_queries',
[]), ensure_ascii=False)}\n\n"
24         "请生成新的补搜计划以填补缺口。"
25     )
26
27     # 调用 Agent
28     payload, content, messages = _invoke_json_agent(
29         state,
30         prompt,
31         agent,
32         agent_name,
33         "reflect",
34         fallback,
35     )
36
37     return {
38         "iteration": state.get("iteration", 0) + 1, # 迭代计数+1
39         "supplementary_queries": payload.get("supplementary_queries",
fallback["supplementary_queries"]),
40         "messages": messages,
41     }

```

3.9 Writer（撰稿人）

3.9.1 职责说明

核心任务：

1. 整合所有分析结论（findings）
2. 生成结构化的 Markdown 深度研报
3. 在正文中使用正确的引用标记（如 [WEB1_1-1]）
4. 确保引用 ID 合法性（通过 `_validate_and_fix_citations`）

输出要求：

- 篇幅：2000-3000字以上

- 结构：标题、摘要、详细分析、总结展望
- 引用：使用 source_index 中的合法 source_id

3.9.2 完整 Prompt

代码块

```
1  "write": ""你是资深研究员与高级智库撰稿人，负责最终深度研报的撰写。
2
3  【输入内容】
4  你会拿到：
5  1. 核心问题
6  2. 子问题拆解
7  3. 分析结论 (findings)
8  4. 可用来源索引 (source_index)
9  5. 合法引用ID列表
10 6. 风险/冲突标记 (audit_flags)
11
12 【撰写要求】
13
14 1. 标题：
15     - 简明扼要，具有洞察力
16     - 体现研究核心发现
17
18 2. 核心摘要（200字左右）：
19     - 总结最重要的发现
20     - 点明研究价值
21
22 3. 详细分析（主体部分，2000-3000字）：
23     - 将每个 finding 展开为长篇连贯的段落
24     - 进行深度剖析、背景补充和逻辑推演
25     - 严禁一笔带过
26     - 引用证据时使用上标如 [WEB1_1-1]、[LOC1_1-3]
27
28 4. 总结与展望：
29     - 核心结论回顾
30     - 风险提示
31     - 未来展望
32
33 【极其重要的警告】
34 - 你的核心任务是扩写和深度分析，必须保证字数充足，绝不能写成简短的大纲或骨架！
35 - 绝对禁止输出任何 JSON 格式、字典结构或大括号（{}）！
36 - 严禁自行编造引用序号（如 [WEB-10]），你只能使用 source_index 中提供的合法
    source_id!
37 - 你的输出将直接面向行业专家和管理层阅读，必须是一篇极其专业的长文！
```



```
38 - 结尾不需要你来列举引用列表，你只需要在正文中打好合法的引用标记即可，系统会自动在文章末尾
    拼接参考资料。
39
40 【输出格式】
41 直接输出 Markdown 格式的报告正文，不要输出 JSON，不要输出代码块标记（``）。"""
```

3.9.3 节点实现代码

代码块

```
1 def _extract_citation_ids(content: str) -> list[str]:
2     """从正文中提取所有引用ID [XXX]"""
3     pattern = r'\([([A-Z]+\d+\_?\d+--?\d+)\)'
4     matches = re.findall(pattern, content)
5     return list(dict.fromkeys(matches)) # 去重保序
6
7 def _validate_and_fix_citations(content: str, valid_source_ids: set[str]) ->
    tuple[str, list[str]]:
8     """校验正文中的引用ID，移除非法引用"""
9     pattern = r'\([([A-Z]+\d+\_?\d+--?\d+)\)'
10
11     def replace_citation(match):
12         citation_id = match.group(1)
13         if citation_id in valid_source_ids:
14             return f"[{citation_id}]"
15         else:
16             # 非法引用，直接移除
17             return ""
18
19     fixed_content = re.sub(pattern, replace_citation, content)
20     used_ids = [cid for cid in _extract_citation_ids(fixed_content) if cid in
    valid_source_ids]
21     return fixed_content, used_ids
22
23 def _render_reference_list(state: ResearchState) -> str:
24     """渲染引用列表"""
25     lines = ["## 参考资料"]
26     lookup = _build_source_lookup(state)
27
28     # 从正文提取实际引用的 source_id
29     draft_content = state.get("draft", "") or state.get("final", "")
30     cited_ids: list[str] = []
31     if draft_content:
```

```
32         for sid in _extract_citation_ids(draft_content):
33             if sid in lookup and sid not in cited_ids:
34                 cited_ids.append(sid)
35
36         # 分离 WEB 和 LOCAL
37         web_ids = [sid for sid in cited_ids if lookup.get(sid,
38 {}).get("source_type") != "local"]
39         local_ids = [sid for sid in cited_ids if lookup.get(sid,
40 {}).get("source_type") == "local"]
41
42         # 去重 (LOCAL 按 locator 去重)
43         seen_locators: set[str] = set()
44         display_ids: list[str] = []
45         for sid in web_ids + local_ids:
46             source = lookup.get(sid)
47             if not source:
48                 continue
49             locator = source.get("locator", "").strip()
50             if source.get("source_type") == "local":
51                 dedup_key = locator or sid
52                 if dedup_key in seen_locators:
53                     continue
54                 seen_locators.add(dedup_key)
55                 display_ids.append(sid)
56
57         # 渲染
58         for sid in display_ids:
59             source = lookup.get(sid)
60             if not source:
61                 continue
62             locator = source.get("locator", "").strip()
63             label = source.get("label", "").strip()
64             source_type = source.get("source_type", "source")
65             source_id = source.get("source_id", sid)
66
67             if not locator:
68                 locator = "链接暂不可用" if source_type == "web" else "本地知识库"
69
70             lines.append(f"- [{source_id}] [{source_type}]: {label} | {locator}")
71
72         if len(lines) == 1:
73             lines.append("- 暂无参考资料")
74         return "\n".join(lines)
75
76 def _ensure_reference_section(content: str, state: ResearchState) -> str:
77     """确保有引用列表"""
78     base = content.rstrip()
```

```

77     references = _render_reference_list(state)
78     if "## 引用列表" in base or "## 来源清单" in base or "## 参考资料" in base:
79         return base
80     return f"{base}\n\n{references}"
81
82 def write_node(state: ResearchState, agent, agent_name: str) -> ResearchState:
83     """写作节点"""
84     logger.info("%s 开始 | agent=%s", colorize("[write]", "cyan"),
85                 colorize(agent_name, "magenta"))
86
87     # 获取合法引用ID
88     valid_source_ids = [str(item.get("source_id", "")).strip()
89                         for item in state.get("source_index", []) if
90                         item.get("source_id")]
91     valid_source_ids = [item for item in valid_source_ids if item][:80]
92     valid_source_ids_set = set(valid_source_ids)
93
94     # 构建 Prompt
95     prompt = (
96         "请严格根据以下信息撰写最终的 Markdown 研报。请直接输出正文，绝对不要输出任何\n\n"
97         f"核心问题: {state['query']}\n\n"
98         f"子问题拆解: {json.dumps(state.get('sub_questions', []),\n\n"
99         ensure_ascii=False)}\n\n"
100         "【分析结论 (Findings)】: \n\n"
101         f"{json.dumps(state.get('findings', []), ensure_ascii=False)}\n\n"
102         "【可用来源索引 (source_index)】: \n\n"
103         f"{json.dumps(state.get('source_index', []), ensure_ascii=False)}\n\n"
104         "【合法引用ID列表】: \n\n"
105         f"{json.dumps(valid_source_ids, ensure_ascii=False)}\n\n"
106         "【可能存在的风险/冲突 (Audit Flags)】: \n\n"
107         f"{json.dumps(state.get('audit_flags', []), ensure_ascii=False)}\n\n"
108         "要求: 正文必须使用合法引用ID (例如 [WEB1_1-1]、[LOC1_1-3]) ; 禁止使用不存在的\n\n"
109         "编号。"
110         "结尾不需要你来列举引用列表，系统会自动拼接。"
111     )
112
113     human = HumanMessage(content=with_memory_context(state, prompt))
114
115     # 只给单条指令，避免被前面的 JSON 带偏
116     result = agent.invoke({"messages": [human]})
117     content = _last_content(result)
118
119     # 强制清理可能的错误 JSON 代码块
120     content = re.sub(r"^```json\s*", "", content)
121     content = re.sub(r"^```markdown\s*", "", content)
122     content = re.sub(r"^```\s*", "", content)

```

```

119     content = re.sub(r"```$", "", content.strip())
120
121     # 校验并修正引用ID, 移除非法引用
122     content, used_citation_ids = _validate_and_fix_citations(content,
valid_source_ids_set)
123
124     # 确保有引用列表
125     final_content = _ensure_reference_section(content, state)
126     emit("write", final_content)
127
128     return {"draft": final_content, "final": final_content, "messages":
[human, result["messages"][-1]]}

```

四、项目目录结构

代码块

1	mult_agents_memory/	
2	— app/	# 主应用代码
3	— backend/	# FastAPI 后端服务
4	— config/	# 配置管理
5	— router/	# API 路由
6	— schemas/	# Pydantic 数据模型
7	— service/	# 业务逻辑服务
8	— mult_agents/	# 多智能体核心模块
9	— memory/	# 记忆系统
10	— base.py	# 记忆基础类型
11	— manager.py	# 记忆管理器
12	— long_term.py	# 长期记忆存储
13	— short_term.py	# 短期记忆存储
14	— utils.py	# 记忆工具函数
15	— rag/	# RAG 检索系统
16	— core.py	# RAG 核心实现
17	— ingest.py	# 文档导入工具
18	— config.py	# 应用配置
19	— graph.py	# LangGraph workflow 定义
20	— main.py	# 主入口和 Agent 构建
21	— nodes.py	# 节点执行逻辑
22	— prompts.py	# Agent 提示词

```
23 | | | └─ state.py          # 状态定义
24 | | | └─ tools.py         # 工具函数
25 | | └─ data/              # 数据存储
26 | | └─ app_main.py        # 应用主入口
27 | └─ main.py              # CLI 入口
28 | └─ config.json          # 配置文件
29 | └─ .env                  # 环境变量
30 | └─ requirements.txt      # 依赖清单
```

五、核心技术栈亮点

5.1 LangGraph 状态机 workflow

技术亮点：使用 `TypedDict` + `Annotated` 实现类型安全的状态管理

代码块

```
1  # state.py - 状态定义核心代码
2  from typing import Annotated, List
3  from typing_extensions import TypedDict
4  from langchain_core.messages import BaseMessage
5  import operator
6
7  class ResearchState(TypedDict):
8      query: str                # 用户原始问题
9      user_id: str              # 用户标识
10     tenant_id: str             # 租户标识
11     memory_context: str        # 跨会话记忆上下文
12     messages: Annotated[List[BaseMessage], operator.add] # 消息累积
13     intent: str                # 意图识别结果
14     phase: str                 # 当前阶段
15     plan: str                  # 规划摘要
16     outline: List[dict]        # 大纲结构
17     sub_questions: List[str]   # 子问题列表
18     research_questions: List[str] # 研究问题
19     search_plan: List[dict]    # 搜索计划
20     budget: dict               # 资源预算
```

```

21 web_search: str # 网络检索结果摘要
22 local_rag: str # 本地检索结果摘要
23 web_evidence: list[dict] # 网络证据列表
24 local_evidence: list[dict] # 本地证据列表
25 evidence_pool: list[dict] # 统一证据池
26 deep_dive: str # 深度分析摘要
27 audit: str # 审计摘要
28 audit_flags: list[dict] # 审计标记
29 analysis: str # 分析结论
30 needs_more_research: bool # 是否需要更多研究
31 missing_gaps: list[str] # 信息缺口
32 supplementary_queries: list[dict] # 补搜查询
33 findings: list[dict] # 研究发现
34 claim_map: list[dict] # 结论-来源映射
35 source_index: list[dict] # 来源索引
36 web_retrieval_stats: dict # 网络检索统计
37 local_retrieval_stats: dict # 本地检索统计
38 web_search_trace: list[dict] # 网络搜索追踪
39 local_rag_trace: list[dict] # 本地RAG追踪
40 draft: str # 报告草稿
41 final: str # 最终报告
42 iteration: int # 当前迭代次数
43 max_iterations: int # 最大迭代次数

```

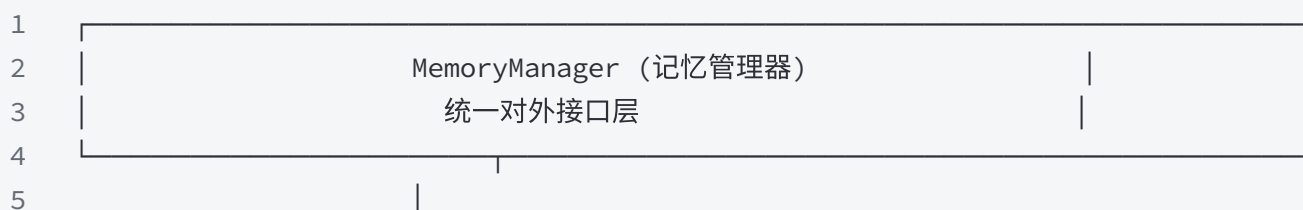
设计亮点：

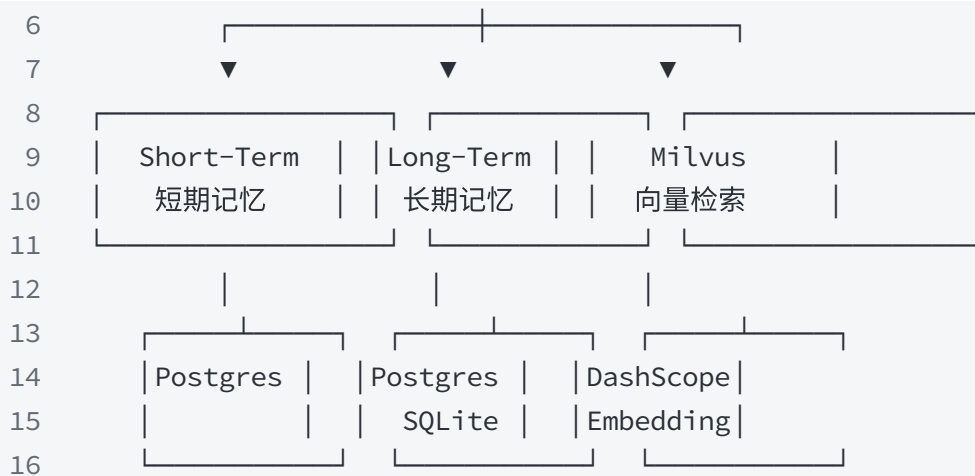
- `Annotated[List[BaseMessage], operator.add]` 实现消息自动累积
- 所有字段类型明确，支持 IDE 智能提示和类型检查
- 状态字段覆盖完整的研究生命周期

5.2 分层记忆系统

架构设计:

代码块





核心代码 (manager.py) :

代码块

```

1  class MemoryManager:
2      def __init__(
3          self,
4          short_term_backend: str = "postgres", # 支持 postgres/memory
5          long_term_backend: str = "postgres", # 支持 postgres/sqlite/disabled
6          enable_milvus: bool = True, # 是否启用向量检索
7          long_term_scope: str = "user", # user 级或 thread 级
8      ):
9          # 三层记忆初始化
10         self.short_term = ShortTermMemory(...) # 会话级
11         self.semantic = SemanticMemoryStore(...) # 语义记忆
12         self.episodic = EpisodicMemoryStore(...) # episodic 记忆
13         self._milvus_store = None # 向量检索
14
15     def build_personalized_prompt_context(self, user_id, thread_id, query):
16         """构建个性化提示上下文 - 核心接口"""
17         # 1. 获取用户画像
18         profile = self.get_user_profile(user_id)
19         # 2. 获取对话摘要
20         summary = self.get_short_term_summary(thread_id)
21         # 3. 语义搜索相关记忆
22         memories = self.search_semantic(query, user_id)
23         # 4. 组装上下文
24         return f"""## 用户画像\n{profile}\n\n## 对话摘要\n{summary}\n\n## 相关记忆\n{memories}"""

```

5.3 智能引用验证系统

问题背景：大模型经常幻觉引用不存在的来源编号

解决方案：

代码块

```
1  # nodes.py - 引用验证核心代码
2
3  def _extract_citation_ids(content: str) -> list[str]:
4      """从正文中提取所有引用ID [XXX]"""
5      pattern = r'\([([A-Z]+\d+_\d+--\d+)\)'
6      matches = re.findall(pattern, content)
7      return list(dict.fromkeys(matches)) # 去重保序
8
9  def _validate_and_fix_citations(content: str, valid_source_ids: set[str]) ->
    tuple[str, list[str]]:
10     """校验正文中的引用ID, 移除非法引用"""
11     pattern = r'\([([A-Z]+\d+_\d+--\d+)\)'
12
13     def replace_citation(match):
14         citation_id = match.group(1)
15         if citation_id in valid_source_ids:
16             return f"[{citation_id}]"
17         else:
18             # 非法引用, 直接移除
19             return ""
20
21     fixed_content = re.sub(pattern, replace_citation, content)
22     used_ids = [cid for cid in _extract_citation_ids(fixed_content) if cid in
    valid_source_ids]
23     return fixed_content, used_ids
```

5.4 多轮迭代研究机制

实现逻辑：

代码块

```
1  # graph.py - 条件路由
2
3  def should_continue_research(state: ResearchState) -> str:
4      iteration = state.get("iteration", 0)
5      max_iter = state.get("max_iterations", 2)
6
7      # 1. 达到最大迭代次数, 停止
8      if iteration >= max_iter:
9          return "write"
10
11     # 2. 分析师发现信息缺口, 需要补搜
12     if state.get("needs_more_research", False):
13         return "reflect"
14
15     # 3. 证据充足, 直接写报告
16     return "write"
17
18 # 工作流连接
19 workflow.add_conditional_edges(
20     "analyze",
21     should_continue_research,
22     {
23         "reflect": "reflect", # 需要补搜
24         "write": "write"     # 直接写报告
25     }
26 )
27 workflow.add_edge("reflect", "web_search") # 补搜回到检索
28 workflow.add_edge("reflect", "local_rag")
```

5.5 双源检索并行融合

并行检索设计:

代码块

```
1  # graph.py
2  workflow.add_edge("plan", "web_search") # 规划后同时触发
3  workflow.add_edge("plan", "local_rag")  # 网络 + 本地并行
```

```
4 workflow.add_edge("web_search", "deep_dive") # 两者都完成后
5 workflow.add_edge("local_rag", "deep_dive") # 才进入证据裁判
```

证据融合与评分：

代码块

```
1 def _score_evidence(record: dict) -> tuple[float, str]:
2     source_type = record.get("source_type")
3     if source_type == "local":
4         return 0.92, "企业内部知识库证据，默认高可信"
5     domain = str(record.get("domain", "")).lower()
6     if _is_official_domain(domain):
7         return 0.88, "官方或权威机构域名"
8     if any(word in domain for word in ["news", "finance", "reuters",
9         "bloomberg"]):
10         return 0.72, "主流媒体域名"
11     if domain:
12         return 0.58, "普通互联网来源，需要交叉验证"
13     return 0.45, "来源信息不完整"
```

六、关键设计模式

6.1 节点绑定模式

代码块

```
1 # 使用 functools.partial 绑定 Agent 到节点函数
2 def bind_agent(node_func, agent, agent_name: str):
3     return partial(node_func, agent=agent, agent_name=agent_name)
4
5 # 构建工作流时绑定
6 workflow.add_node("plan", bind_agent(plan_node, agents.planner, "planner"))
```

```
7 workflow.add_node("web_search", bind_agent(web_search_node, agents.scout_web,
"scout_web"))
```

6.2 JSON 安全解析模式

代码块

```
1 def _extract_json_block(text: str) -> str:
2     """提取代码块中的 JSON"""
3     cleaned = text.strip()
4     if cleaned.startswith("`"):
5         cleaned = re.sub(r"^`(?:json)?", "", cleaned).strip()
6         cleaned = re.sub(r"`$", "", cleaned).strip()
7     start = cleaned.find("{")
8     end = cleaned.rfind("}")
9     if start != -1 and end != -1 and end > start:
10         return cleaned[start : end + 1]
11     return cleaned
12
13 def _load_json(text: str, fallback: dict) -> dict:
14     """安全加载 JSON, 失败返回 fallback"""
15     try:
16         value = json.loads(_extract_json_block(text))
17         if isinstance(value, dict):
18             return value
19     except Exception:
20         pass
21     return fallback # 每个节点都有 fallback, 确保系统不崩溃
```

6.3 来源去重模式

代码块

```
1 def _dedupe_sources(items: list[dict], key_fields: list[str]) -> list[dict]:
2     """基于指定字段去重"""
3     seen = set()
4     results = []
5     for item in items:
```

```
6         key = tuple(str(item.get(field, "")).strip() for field in key_fields)
7         if key in seen:
8             continue
9         seen.add(key)
10        results.append(item)
11    return results
12
13    # 使用示例
14    raw_records = _dedupe_sources(raw_records, ["url", "title"])    # 网络证据去重
15    raw_records = _dedupe_sources(raw_records, ["doc_id", "snippet"])    # 本地证据去重
```

七、配置说明

7.1 环境变量 (.env)

代码块

```
1  DASHSCOPE_API_KEY=sk-xxx          # 通义千问 API Key
2  BOCHA_API_KEY=sk-xxx              # Bocha 搜索 API Key
3  POSTGRES_DSN=postgresql://user:pass@host:5432/db
4  MILVUS_HOST=xxx.xxx.xxx.xxx       # Milvus 向量数据库地址
5  MILVUS_PORT=19530
6  MILVUS_COLLECTION=mult_agent_memory
```

7.2 配置文件 (config.json)

代码块

```
1  {
2    "api_key": "",
3    "model": "qwen-turbo",
4    "max_iterations": 3,
5    "enable_memory": true,
6    "short_term_backend": "postgres",
7    "long_term_backend": "postgres",
8    "enable_milvus": true,
9    "checkpoint_backend": "postgres"
```

```
10 }
```

八、运行方式

8.1 CLI 模式

代码块

```
1  # 单次查询
2  python main.py --user-id user03 --thread-id th --once-query "帮我调查2026年最好的
   AI产品"
3
4  # 交互模式
5  python main.py --user-id user03 --thread-id th
6
7  # 参数覆盖
8  python main.py --user-id user03 --thread-id th \
9      --short-term-backend postgres \
10     --long-term-backend postgres \
11     --enable-milvus true
```

8.2 API 服务模式

代码块

```
1  # 启动 FastAPI 服务
2  python -m app.app_main
3
4  # 或
5  uvicorn app.app_main:app --host 0.0.0.0 --port 8000
```